

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	C01 – Imitation (L1)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
C01 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	K.Ramesh	Reg. No.:	192472385
Section / Batch:	CSE AI	Date:	13/07/2026

Code of Conduct

I K. Ramesh (192472385)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: __K.Ramesh_____

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Annexure A – Application Based Questions

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.

Identify email addresses.

Extract mobile numbers.

Detect technical skills (Python, Java, SQL, Machine Learning, NLP).

Extract years of experience.

Generate a structured summary of the candidate's profile.

Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.

Perform prefix-based searches.

Perform suffix-based searches.

Search using partial keywords.

Perform case-insensitive searches.

Display all matching product names.

Generate a report showing the total number of matching products for each search.

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.

Validate the institutional email address.

Validate the course code.

Validate the semester information.

Validate the student's mobile number.

Display appropriate validation messages for each field.

Generate a final registration status report indicating whether the registration is successful.

Section 1) Aim:

To develop a Python program using Regular Expressions (Regex) to extract candidate details from a resume and identify eligible candidates.

1. Procedure:

Import the re module.

Read the resume text.

Extract the name, email, mobile number, skills, and experience using Regex.

Check if the candidate has Python skill and at least 2 years of experience.

Display the extracted details and eligibility status.

Code:

```
import re
```

```
resume = """
```

```
Name: Ramesh Kumar
```

```
Email: ramesh@gmail.com
```

```
Mobile: 9876543210
```

```
Skills: Python, Java, SQL
```


Experience: 3 years

"""

```
name = re.search(r"Name:\s*(.*)", resume).group(1)
```

```
email = re.search(r"\S+@\S+\.\S+", resume).group()
```

```
mobile = re.search(r"\d{10}", resume).group()
```

```
exp = int(re.search(r"(\d+)\s+years", resume).group(1))
```

```
skills = []
```

```
for skill in ["Python", "Java", "SQL", "Machine Learning", "NLP"]:
```

```
    if re.search(skill, resume, re.I):
```

```
        skills.append(skill)
```

```
print("Name:", name)
```

```
print("Email:", email)
```

```
print("Mobile:", mobile)
```

```
print("Skills:", skills)
```

```
print("Experience:", exp, "Years")
```

```
if exp >= 2 and "Python" in skills:
```

```
    print("Eligible")
```

```
else:
```

```
    print("Not Eligible")
```

Output:

Name: Ramesh Kumar

Email: ramesh@gmail.com

Mobile: 9876543210

Skills: ['Python', 'Java', 'SQL']

Experience: 3 Years

Eligible

Result:

The program successfully extracted resume details using **Regular Expressions** and identified the eligible candidate based on the given criteria.

Section 2) Aim:

To develop a Python program using Regular Expressions (Regex) to perform flexible product searches and display matching products.

Procedure:

Import the **re** module.

Enter the product list and search keyword.

Perform exact, prefix, suffix, partial, and case-insensitive searches using Regular Expressions.

Count and display the total number of matches for each search.

Stop.

Code:

```
import re
```

```
products = [
```

```
    "Apple iPhone",
```

```
    "Apple Watch",
```

```
    "Samsung Galaxy",
```

```
    "Galaxy Buds",
```

```
    "Dell Laptop",
```

```
    "HP Laptop",
```

```
    "Wireless Mouse",
```

"Gaming Mouse"

]

keyword = "Apple"

Exact Search

exact = [p for p in products if re.fullmatch(keyword, p, re.I)]

Prefix Search

prefix = [p for p in products if re.match(keyword, p, re.I)]

Suffix Search

suffix = [p for p in products if re.search(keyword + r"\$", p, re.I)]

Partial Search

partial = [p for p in products if re.search(keyword, p, re.I)]

print("Exact Search:", exact)

print("Prefix Search:", prefix)

print("Suffix Search:", suffix)

print("Partial Search:", partial)

print("\nReport")

print("Exact Matches:", len(exact))

print("Prefix Matches:", len(prefix))

print("Suffix Matches:", len(suffix))

print("Partial Matches:", len(partial))

Output:

Exact Search: []

Prefix Search: ['Apple iPhone', 'Apple Watch']

Suffix Search: []

Partial Search: ['Apple iPhone', 'Apple Watch']

Report

Exact Matches: 0

Prefix Matches: 2

Suffix Matches: 0

Partial Matches: 2

Result:

The program successfully performed exact, prefix, suffix, partial, and case-insensitive product searches using Regular Expressions and displayed the matching products along with the total number of matches.

Section 3) Aim:

To develop a Python program using Regular Expressions (Regex) to validate student registration details and generate the registration status.

Procedure:

Import the re module.

Enter the student registration details.

Validate the register number using a Regular Expression.

Validate the course code.

Validate the semester information.

Validate the mobile number.

Display validation messages for each field.

Display the final registration status (Successful or Failed).

Stop.

Code:

```
import re
```

```
reg_no = "22CSE101"
```

```
email = "student@university.edu"

course = "CS301"

semester = "5"

mobile = "9876543210"

reg = re.fullmatch(r"\d{2}[A-Z]{3}\d{3}", reg_no)

mail = re.fullmatch(r"[a-zA-Z0-9._%+-]+@university\.edu", email)

code = re.fullmatch(r"[A-Z]{2}\d{3}", course)

sem = re.fullmatch(r"[1-8]", semester)

phone = re.fullmatch(r"\d{10}", mobile)

print("Register Number:", "Valid" if reg else "Invalid")

print("Email:", "Valid" if mail else "Invalid")

print("Course Code:", "Valid" if code else "Invalid")

print("Semester:", "Valid" if sem else "Invalid")

print("Mobile:", "Valid" if phone else "Invalid")

if reg and mail and code and sem and phone:

    print("\nRegistration Successful")

else:

    print("\nRegistration Failed")
```

Output:

Register Number: Valid

Email: Valid

Course Code: Valid

Semester: Valid

Mobile: Valid

Registration Successful

Result:

The program successfully validated the student registration details using Regular Expressions and displayed the final registration status.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____